
DEVELOPER AND USER HANDBOOK

polestar-mcp

The Model Context Protocol server for the Polestar Data Portal M2M API. This handbook covers setup from a blank machine, all 23 read-only tools (26 with history), the architecture behind them, and the verification gates that keep the project honest.

M2M API v1 · read-only · 23 tools + history

POLESTAR-MCP · DATA PORTAL M2M

Contents

1. Getting started	3
What this is	3
What you need	3
Step 1: Get a Data Portal credential	3
Step 2: Store the credential outside the project	4
Step 3: Build and register the server	4
All environment variables	5
First calls	6
History, sampling, and privacy	7
HTTP mode	7
Trying it without a Polestar credential	7
Troubleshooting	8
2. Tool reference	9
How tools map to the API	9
The domain tools (15 + list_vehicles)	10
Aggregate tools (4)	12
Advisor tool (1)	13
System tool (1)	13
History tools (3, opt-in)	13
Resources, prompts, and annotations	13
When data is missing: DATA_NOT_AVAILABLE	14
3. Architecture	14
The mental model in one paragraph	14
What each module does	15
Error model	19
Design decisions, and why	19
4. The upstream API contract	20
What the M2M API is	20
Base URL and shape	20
Authentication	20
Request headers	21

Response envelope	21
The availability model	21
Error shape	22
Endpoints used	22
Write scopes without write endpoints	22
What the server adds on top of the contract	23
5. Development and verification	23
Repository layout	24
Commands	25
What the unit tests prove	25
End-to-end scripts	26
The verification gates	26
Secret hygiene	27
Adding a tool	27
Versioning	28
6. Live contract findings: Polestar Data Portal M2M	28
Identity headers (settles E7)	28
VIN handling (settles Q3)	28
Domain list (settles Q5 partially)	29
Live domain scan, single authorized vehicle	29
Error envelope shape	29
Token endpoint	29
No published contract at the gateway	30
7. Consumer API adapter design note (not implemented)	30
Why it is not wired in	30
Shape when it lands	30
8. Write endpoints (designed, waiting for Polestar)	31
What slots in where	31
Safety design (non-negotiable when writes land)	31

Getting started

Who this is for: anyone who wants to ask an AI assistant about their Polestar. No programming needed beyond pasting a few commands. Developers will find the internals in `architecture.md` and the full tool list in `tools-reference.md`.

What this is

`polestar-mcp` is a server that connects an MCP client (Claude Desktop, ZCode, or any other tool that speaks the Model Context Protocol) to the Polestar Data Portal M2M API. Once it is connected, you can ask your assistant things like "how is my car doing?" or "is everything locked?" and it will call one of the server's tools to get the answer from Polestar's servers.

The whole surface is read-only: every tool carries the MCP `readOnlyHint`, and nothing is ever sent to the car. That is a property of the API Polestar publishes today, not a choice this server made (see `upstream-api.md`).

Because the API allows only 10,000 calls per client per day, the server manages that budget for you: repeated reads are served from a cache, every live call is metered, and the server refuses (rather than fails mid-answer) once the day's budget is spent. A `polestar_status` tool lets your assistant check what is left.

What you need

- Node.js 20 or newer (`node --version` to check).
- An MCP client, that is, an app that can connect to MCP servers.
- One of:
 - a **Polestar Data Portal credential** (for live data), or
 - **nothing at all** (for fixture mode, which replays a previously captured snapshot of responses so you can try the server offline).

Step 1: Get a Data Portal credential

1. Log in at data-portal.polestar.com.
2. Open the **Data Portal API** tab.
3. Create a credential. Three values matter, and the page shows all of them:
 - the **client ID**,
 - the **client secret** (shown once, so store it somewhere safe),
 - the **Account ID**, also labeled **x-client-id**, shown near the top of the page.

The Account ID and the OAuth client ID are two different identifiers, and the API needs both. This is worth getting right: the gateway rejects every request whose `x-client-id` carries the OAuth client ID instead of the Account ID (error 403 `AUTHZ_CLIENT_ID_MISMATCH`), and the server warns at startup when it detects the two

values are identical. See `upstream-api.md` for where each one is used.

Step 2: Store the credential outside the project

The server reads a small dotenv-style file that lives in your home directory, deliberately outside any repository or download folder. Real environment variables always win over the file, and the file only fills in the blanks.

```
mkdir -p ~/.config/polestar-mcp
cat > ~/.config/polestar-mcp/.env.secrets <<'EOF'
POLESTAR_CLIENT_ID=paste-your-client-id-here
POLESTAR_CLIENT_SECRET=paste-your-client-secret-here
POLESTAR_ACCOUNT_ID=paste-your-account-id-here
EOF
```

Keep that file private. It is the equivalent of a password for your car's data. The loader handles the usual dotenv idioms: a leading `export` statement, values in either kind of quote, and trailing `#` comments (a `#` inside a quoted value stays data).

If you would rather keep the file somewhere else, set the `POLESTAR_ENV_FILE` environment variable to its full path. The file is never looked for inside the project tree on purpose, and in fixture mode it is not read at all, so offline use never touches a live credential file.

Step 3: Build and register the server

From the `polestar-mcp` folder:

```
npm install
npm run build
```

Then add an entry like this to your MCP client's configuration (in Claude Desktop this is `claude_desktop_config.json`, in other clients look for "MCP servers" in their settings):

```
{
  "mcpServers": {
    "polestar": {
      "command": "node",
      "args": ["/absolute/path/to/polestar-mcp/build/server.js"],
      "env": {
        "POLESTAR_CLIENT_ID": "<client id>",
        "POLESTAR_CLIENT_SECRET": "<client secret>",
        "POLESTAR_ACCOUNT_ID": "<account id / x-client-id>"
      }
    }
  }
}
```

The `env` block is optional if you created the secrets file in step 2. Use one or the other. Restart your MCP client afterwards so it picks up the new server.

All environment variables

The server knows exactly which `POLESTAR_*` variables exist. When you set one it does not recognize, it warns on startup (and suggests the closest real name, so a typo is visible); with `POLESTAR_STRICT_ENV=1` an unknown variable is fatal instead. A value that is present but unusable (a bad URL, a non-numeric timeout, an out-of-range port) fails fast with an explanation before any request is made.

Variable	Default	Purpose
<code>POLESTAR_CLIENT_ID</code>	(required live)	OAuth client ID from the Data Portal
<code>POLESTAR_CLIENT_SECRET</code>	(required live)	OAuth client secret from the Data Portal
<code>POLESTAR_ACCOUNT_ID</code>	(required live)	The Account ID, sent as <code>x-client-id</code> on every vehicle request. Distinct from the OAuth client ID
<code>POLESTAR_DELEGATED_ACCOUNT_ID</code>	unset	Third-party credentials only: default account whose shared vehicles you access (<code>x-delegated-account-id</code>)
<code>POLESTAR_DELEGATED_ACCOUNT_IDS</code>	unset	Semicolon-, comma-, or space-separated allowlist of delegated account IDs a call may select with its <code>delegated_account_id</code> argument. A requested ID outside the list is refused (<code>DELEGATION_NOT_ALLOWED</code>) instead of sent, because the API silently accepts unknown delegated IDs
<code>POLESTAR_BASE_URL</code>	production URL	Override the API base URL. The default is <code>https://pc-api.polestar.com/eu-north-1/data-portal/m2m</code>
<code>POLESTAR_M2M_TOKEN_ENDPOINT</code>	{base URL}/token	Override just the token endpoint
<code>POLESTAR_FIXTURES_DIR</code>	unset	Fixture mode: serve captured responses from this directory instead of calling the real API
<code>POLESTAR_ENV_FILE</code>	unset	Full path to a dotenv file to load, instead of <code>~/.config/polestar-mcp/.env.secrets</code>
<code>POLESTAR_TIMEOUT_MS</code>	15000	Hard deadline per HTTP request
<code>POLESTAR_BUDGET</code>	10000	Daily live-call ceiling. The server fails closed at the limit
<code>POLESTAR_CACHE</code>	on	off (or disabled, 0, false) disables the response cache
<code>POLESTAR_SCOPES</code>	all domain scopes	Space- or comma-separated scope override for the token request
<code>POLESTAR_HISTORY_DIR</code>	unset	Enable the local history store and background sampler in this directory

Variable	Default	Purpose
POLESTAR_HISTORY_SAMPLE_SECONDS	600	Seconds between sampler rounds (clamped 60 to 3600)
POLESTAR_SAMPLE	off	1 (or <code>true</code> , <code>on</code>) actually runs the background sampler. Off by default: polling spends the shared quota
POLESTAR_VEHICLE_LABELS	unset	Semicolon-separated <code>VIN=Friendly Name</code> pairs; output uses the name instead of the VIN
POLESTAR_REDACT_VIN	off	1 (or <code>true</code> , <code>on</code>) masks VINs in tool output (first 3 and last 3 characters only)
POLESTAR_ALLOW_LOCATION	on	0 (or <code>false</code> , <code>off</code> , <code>disabled</code>) makes <code>get_location</code> refuse and strips coordinates from aggregates
POLESTAR_UNITS	km	mi (or miles) for humanized distances
POLESTAR_LOG	normal	debug enables verbose stderr diagnostics
POLESTAR_ENABLE_WRITES	off	Scaffold for future write endpoints; registers nothing today
POLESTAR_HTTP_PORT	0 (stdio)	1 to 65535 serves the same surface over Streamable HTTP instead of stdio
POLESTAR_HTTP_HOST	127.0.0.1	HTTP bind address
POLESTAR_HTTP_TOKEN	unset	Bearer token for HTTP mode; required when binding anything but loopback
POLESTAR_STRICT_ENV	off	1 (or <code>true</code> , <code>on</code>) makes an unrecognized variable fatal

First calls

Ask your assistant something natural. Behind the scenes it will pick the right tool:

- "What cars can I see?" calls `list_vehicles`.
- "How is my car doing?" calls `get_car_status`, a one-call briefing over charge, range, locks, odometer, and service.
- "Is everything locked?" calls `is_car_secure`, which names anything open.
- "When will it be charged?" calls `get_charging_estimate`.
- "Plan the cheapest charging for tonight" calls `plan_cheapest_charge` after you paste hourly prices from your tariff provider.

The full list of tools, with example responses, is in `tools-reference.md`. Three behaviors worth knowing before you start:

- **Missing data is not an error.** A Polestar reports each topic of data independently, and some cars simply do not report some topics. When you ask for a topic the car does not report (most often `get_is_at_charge_location`), the tool answers with a plain sentence saying no data is available for that vehicle.
- **Every answer carries a data age.** The cloud may re-serve a value the car reported minutes (or, for charging settings, days) ago. The age line tells you how fresh the answer is before you trust it.
- **One credential, several cars.** With several VINs on the credential, tools ask for an explicit `vin` rather than guessing; with exactly one, you can omit it everywhere.

History, sampling, and privacy

Set `POLESTAR_HISTORY_DIR` to a directory and the server records every live domain answer to an append-only store there, deduplicated by the cloud's change ID so unchanged snapshots cost a single line. Passive recording costs nothing extra: it reuses answers the server already fetched. If you also want the car sampled while no client is asking, set `POLESTAR_SAMPLE=1`, which runs a background round over the fast-moving domains (battery, odometer, location, exterior) every 10 minutes by default (`POLESTAR_HISTORY_SAMPLE_SECONDS` tunes this) and stands down entirely once 80% of the daily budget is spent. Sampling is off by default because polling spends the shared quota. The history tools (degradation trend, charging sessions) only exist when history is on.

Three privacy dials are built in: `POLESTAR_VEHICLE_LABELS` gives your cars friendly names (`POLESTAR_VEHICLE_LABELS="YSMTEST22PL000001=My Car"`), so answers say "My Car" instead of printing identifiers; `POLESTAR_REDACT_VIN=1` masks VINs in all output (first 3 and last 3 characters only); and `POLESTAR_ALLOW_LOCATION=0` makes the server refuse location reads and strip coordinates from aggregate answers.

HTTP mode

Set `POLESTAR_HTTP_PORT` (for example 8420) and the server serves the same tools, resources, and prompts over stateless Streamable HTTP instead of stdio. It binds `127.0.0.1` by default; binding any other address requires `POLESTAR_HTTP_TOKEN`, because the server reads vehicle location and an unauthenticated network listener would be a real risk. Subscriptions remain a stdio feature.

Trying it without a Polestar credential

Fixture mode answers every tool call from a directory of captured JSON responses instead of the network. It still runs the normal token flow against a captured token response, so what you see is what live use looks like, just with recorded data. The captured dump sits one level above the project, so from inside `polestar-mcp` you can start it like this:

```
POLESTAR_FIXTURES_DIR=.. npm start
```

or run the scripted demo, which asserts real values from that dump and prints `DUMP_OK`:

```
npm run demo-dump
```

A dump without a token response fails with `AuthError: Token request failed (HTTP 404)`. That is intentional: the server does not silently skip authentication in fixture mode.

Troubleshooting

What you see	What it means	What to do
Missing environment variables: <code>POLESTAR_CLIENT_ID</code> , ...	The server found neither environment variables nor a secrets file (note that <code>POLESTAR_ACCOUNT_ID</code> is required too, not optional)	Create the secrets file from step 2, or fill the <code>env</code> block in your client config
Invalid <code>POLESTAR_...: ...</code> at startup	A variable was set but cannot be used (bad URL, non-numeric number, out-of-range port)	Follow the guidance in the message; fix or remove the variable
unrecognized variable: <code>POLESTAR_...</code> (did you mean ...?)	You set a variable the server does not read, so it has no effect	Fix the name (the suggestion is usually right). With <code>POLESTAR_STRICT_ENV=1</code> this exits instead of warning
<code>POLESTAR_ACCOUNT_ID</code> equals <code>POLESTAR_CLIENT_ID</code> warning	The gateway rejects the OAuth client ID as <code>x-client-id</code> with 403 <code>AUTHZ_CLIENT_ID_MISMATCH</code> on every call	Set <code>POLESTAR_ACCOUNT_ID</code> to the Account ID shown next to the Base URL in the Data Portal
<code>AuthError: Token request failed (HTTP 401)</code>	The client ID or client secret is wrong	Re-create the credential on the Data Portal and update the secrets file
<code>AuthError: Token request failed (HTTP 404)</code>	The token endpoint was not found: a wrong base URL, or a fixture dump without a token response	Check <code>POLESTAR_BASE_URL</code> ; in fixture mode make sure <code>token.json</code> exists in the dump
Daily API budget exhausted: ...	The 10,000 calls/day ceiling was reached; the server fails closed instead of hammering the API	Wait for the midnight UTC reset, or raise <code>POLESTAR_BUDGET</code> only if you know your allowance differs
<code>AMBIGUOUS_VIN</code>	The credential sees several cars and no <code>vin</code> was passed	Run <code>list_vehicles</code> and pass an explicit VIN
<code>INVALID_VIN</code>	The VIN passed is not a plausible VIN (17 characters, no I/O/Q)	Check for a truncated or typoed VIN
Request to ... did not complete within ...ms	Polestar did not answer within the deadline	Retry later; the deadline is <code>POLESTAR_TIMEOUT_MS</code> (default 15 seconds)
A tool answers "No ... data is available for vehicle ..."	The car does not report that topic. This is a normal answer, not a failure	Ask for a different topic, or check <code>tools-reference.md</code>
<code>NOT_FOUND</code> errors in fixture mode	A fixture file is missing or misspelled for a route the server knows	This is deliberate: a missing fixture must not masquerade as "the car reports nothing". Fix the dump layout

What you see	What it means	What to do
Refusing to serve on ... without POLESTAR_HTTP_TOKEN	HTTP mode tried to bind a non-loopback address without authentication	Keep 127.0.0.1, or set POLESTAR_HTTP_TOKEN and send Authorization: Bearer <token>
The server never appears in your client	The client could not start build/server.js	Run npm run build, check the absolute path in your client config, and look at the client's MCP logs. The server writes its diagnostics to stderr only, because stdout carries the protocol

One habit worth keeping: the API allows 10,000 calls per client per day. Interactive questions use calls at a human pace, and the cache absorbs repeats, but a script that polls the car in a tight loop will exhaust the daily allowance. The server meters every live call it makes (including retries) and refuses to spend the last call recklessly; `polestar_status` shows exactly where the budget stands.

Tool reference

Who this is for: everyone. Users can skim the tables to see what questions their assistant can answer. Developers will find the exact endpoint, OAuth scope, and output shape behind each tool.

There are 23 tools registered on every start, and 26 with history enabled (three history tools join when the server runs with a history directory). All of them are reads: every tool carries the MCP `readOnlyHint` annotation, so clients can auto-approve calls without prompting you. There is no tool that changes anything on the car, because the documented v1 API publishes no write endpoints (see `upstream-api.md` and the `write-endpoints` design note in `WRITE-ENDPOINTS.md`).

How tools map to the API

Each `get_*` tool wraps exactly one GET endpoint of the Polestar Data Portal M2M API. The naming rule is mechanical: for an API domain named `target-soc`, the tool is `get_target_soc`. The tool's description (what your assistant reads when deciding which tool to call) includes the endpoint path and the OAuth scope, so the mapping stays visible to the AI client too.

The 15 domain tools (one for each of the 15 read domains of the v1 API) are table-driven: one `DOMAINS` entry per read domain, from which the tool name, registration, scope collection, and resource URIs all derive. The aggregate, advisor, system, and history tools are hand-registered in their own modules and read the same data through the client (so they share the cache and the daily budget).

Two arguments are shared by most tools:

Argument	Type	Meaning
vin	string, optional	The Vehicle Identification Number, case-insensitive. Omit it on a credential with exactly one car and that car is used; with several cars the tool asks you to pick (from <code>list_vehicles</code>)
raw	boolean, optional	Domain tools only: append the untouched API payload after the humanized summary

All tools return text content. Domain reads answer with a short humanized summary plus a data age ("data observed 5 min ago"); the raw JSON is one `raw: true` away.

The domain tools (15 + `list_vehicles`)

Tool	Endpoint (after the base URL)	OAuth scope
<code>list_vehicles</code>	GET <code>/v1/vehicles</code>	(none, the token call carries all scopes)
<code>list_domains</code>	(reads the server's own registry)	(none)
<code>get_availability</code>	GET <code>/v1/vehicles/{vin}/telemetry/availability</code>	pdp-telemetry/availability
<code>get_battery</code>	GET <code>/v1/vehicles/{vin}/telemetry/battery</code>	pdp-telemetry/battery
<code>get_exterior</code>	GET <code>/v1/vehicles/{vin}/telemetry/exterior</code>	pdp-telemetry/exterior
<code>get_health</code>	GET <code>/v1/vehicles/{vin}/telemetry/health</code>	pdp-telemetry/health
<code>get_location</code>	GET <code>/v1/vehicles/{vin}/telemetry/location</code>	pdp-telemetry/location
<code>get_odometer</code>	GET <code>/v1/vehicles/{vin}/telemetry/odometer</code>	pdp-telemetry/odometer
<code>get_parking_climatization</code>	GET <code>/v1/vehicles/{vin}/telemetry/parking-climatization</code>	pdp-telemetry/parkingClimatization
<code>get_pre_cleaning</code>	GET <code>/v1/vehicles/{vin}/telemetry/pre-cleaning</code>	pdp-telemetry/preCleaning
<code>get_amp_limit</code>	GET <code>/v1/vehicles/{vin}/charging/amp-limit</code>	pdp-charging/ampLimit
<code>get_charge_locations</code>	GET <code>/v1/vehicles/{vin}/charging/charge-locations</code>	pdp-charging/chargeLocations
<code>get_charge_now</code>	GET <code>/v1/vehicles/{vin}/charging/charge-now</code>	pdp-charging/overrideChargeTimer

Tool	Endpoint (after the base URL)	OAuth scope
<code>get_global_charge_timer</code>	GET /v1/vehicles/{vin} /charging /global-charge-timer	pdp-charging/globalChargeTimer
<code>get_parking_climate_timer</code>	GET /v1/vehicles/{vin} /charging /parking-climate-timer	pdp-charging /parkingClimateTimer
<code>get_target_soc</code>	GET /v1/vehicles/{vin} /charging/target-soc	pdp-charging/targetSoc
<code>get_is_at_charge_location</code>	GET /v1/vehicles/{vin} /charging /is-at-charge-location	pdp-charging /isAtChargeLocation

`list_domains` needs no vehicle and no network call: it prints the server's own registry, every domain with its endpoint, OAuth scope, and tool name. Its description tells the assistant to answer domain questions from here rather than guessing a name, because a mistyped domain is a 404 `VALIDATION_RESOURCE_NOT_FOUND` from the API, which is a different thing from a vehicle reporting no data.

One naming quirk to be aware of: `get_charge_now` reads the charge-now override state, and its scope is `pdp-charging/overrideChargeTimer`, not a scope named after charge-now. That is how Polestar scopes it, and the server keeps the API's vocabulary rather than inventing its own.

What each domain contains:

- `get_availability`: whether the car is currently reachable, the availability status and usage mode. A sleeping car may be listed but not awake.
- `get_battery`: the richest domain. Charge level in percent, charging status, charger connection and power status, charging volts, amps and watts, estimated distance to empty, estimated time to full, average consumption, and the usable energy (`dischargeInfo.energyAvailable`) that feeds the charging estimate and planner.
- `get_exterior`: doors, windows, hood, tailgate and tank lid open or closed, central lock, tailgate lock, and alarm state.
- `get_health`: service countdown in days, distance, and engine hours, the service warning flag, and fluid and light warnings: brake fluid, coolant, oil, washer fluid, and 19 exterior light positions.
- `get_location`: last known latitude, longitude, altitude, heading, and speed, with a timestamp.
- `get_odometer`: odometer in meters, trip meters (manual and automatic) in km, and average speeds in km/h.
- `get_parking_climatization`: whether parking climate (preheat or pre-cool) is running, runtime left, and the requested seat and steering-wheel heating intensities and ventilation.
- `get_pre_cleaning`: cabin air pre-cleaning status, measured PM2.5 and air quality index, and the validity and completion time of the last cycle.
- `get_amp_limit`: the configured maximum AC charging current in amps, its source, and when it was last updated.
- `get_charge_locations`: the charge locations saved for the vehicle.

- `get_charge_now`: the charge-now override state, that is, the override charge timer sync status.
- `get_global_charge_timer`: the recurring charge window: start and stop hour with timezone offset, activation flag, pending changes, and sync status.
- `get_parking_climate_timer`: scheduled parking-climate timers: ready-at hour, weekdays, repeat flag, and activation.
- `get_target_soc`: the target battery state of charge in percent, the setting type (for example `CUSTOM` or `PRESET`), and the last update time.
- `get_is_at_charge_location`: whether the car is currently at a saved charge location. This is the domain most often unsupported by a vehicle.

Example: `get_battery` (humanized)

This is what a domain answer looks like, rendered from the project's sanitized test fixture (synthetic values):

```
Vehicle YSMTEST22PL000001: telemetry/battery (data observed 12 min ago)
Charge: 52% · range 170 km
Charging: Idle · Unplugged
Average consumption: 20.1 kWh/100km
```

With `raw: true` the untouched payload is appended after the summary, so nothing the humanizer does not yet cover is ever lost.

Aggregate tools (4)

The aggregates exist because the questions people actually ask span several domains. Each one reads its constituent domains through the cache, so it costs at most one live call per constituent and usually zero.

- `get_car_status`: a one-call briefing: charge level and state, range, what is open and locked, availability, odometer, service countdown, and the newest data age across the five domains it reads. Use this instead of five separate `get_*` calls.
- `is_car_secure`: a true/false security check. Names anything open (doors, windows, hood, tailgate), whether the central lock is engaged, and the alarm state.
- `get_needs_attention`: only the active warnings from the health domain (fluids, lights, 12V, service) plus the service countdown. An empty answer means nothing needs attention.
- `get_charging_estimate`: energy and time needed to reach the target charge level. Uses the car's live charging power when plugged in, otherwise the configured amp limit times your AC phases (default 3), and derives pack capacity from the reported usable energy when available. Every assumption is stated in the answer.

Advisor tool (1)

- `plan_cheapest_charge`: the read-only smart-charging advisor. You supply an hourly spot-price curve as an argument (so no external service is contacted); the tool combines it with the car's current charge level, target level, amp limit, and the charge window configured in the car's global charge timer, and computes the cheapest contiguous hours that deliver enough energy to reach the target. If the window is too short it falls back to the cheapest individual hours and says so. It never sends anything to the car.

System tool (1)

- `polestar_status`: the server's self-report: daily API budget (used, limit, and when it resets), cache hit rate, token expiry, and mode. Call it when you suspect the daily API budget is running low. See `getting-started.md` for why the budget exists.

History tools (3, opt-in)

Registered only when the server runs with a history directory (`POLESTAR_HISTORY_DIR`). They answer the longitudinal questions the stateless API cannot:

- `get_history`: raw recorded samples for one domain, one line per change, newest first. Duplicates (same cloud `metaEventId`) are never stored twice.
- `get_degradation_report`: the trend of the battery's implied full-charge range (range divided by SoC) and implied pack capacity over recorded history, presented as a series of estimates rather than a single verdict. Needs at least 5 recorded samples.
- `get_charging_sessions`: charging sessions reconstructed from recorded battery samples: start, duration, peak power, and the maximum charge level reached.

Resources, prompts, and annotations

Beyond tools, the server exposes:

- **Resources**: `polestar://vehicle/{vin}/{telemetry|charging}/{domain}`. The list is derived from the credential's vehicles and the domain registry, VIN and domain arguments support completions, and a resource read returns the same humanized view as the matching tool.
- **Subscriptions**: a client can subscribe to a resource URI. The server re-reads it once a minute through the cache and notifies the client only when the cloud's `metaEventId` changed, which happens only when the car actually reported something new. Subscriptions pause rather than spend the last 10% of the daily budget.
- **Prompts**: three starter prompts (`car-status`, `charge-plan`, `battery-health-report`) that wire common questions to the right tool sequence.

When data is missing: DATA_NOT_AVAILABLE

A Polestar reports each domain independently, and "not reported" is a normal state, not a fault. If a car reports nothing at all for a domain, the API answers with HTTP 404 and the error code `DATA_NOT_AVAILABLE`, and the tool turns that into a plain sentence instead of an error result:

```
No is-at-charge-location data is available for vehicle YSMTEST22PL000001
(the API returned DATA_NOT_AVAILABLE: this vehicle does not report this domain).
```

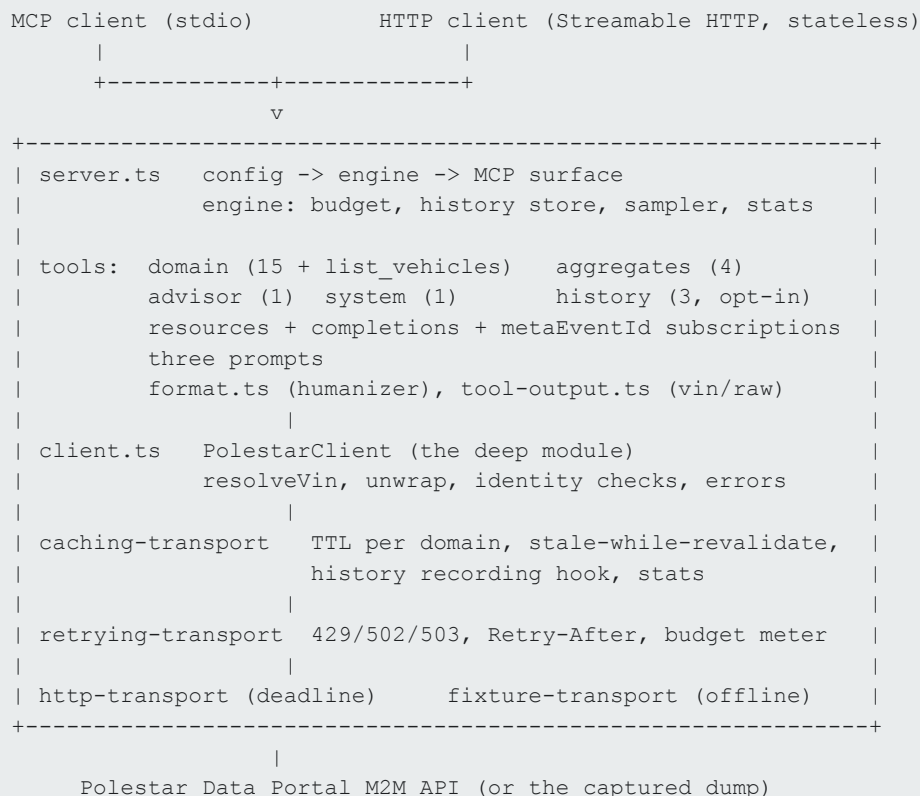
Your assistant will read that sentence and can tell you the car does not report the topic. This is why asking "is my car at a charging station?" sometimes has no answer: it depends on whether that car reports the domain at all.

Architecture

Who this is for: developers who want to change the server, review it, or understand why it is built the way it is. For setup instructions see `getting-started.md`; for the API contract see `upstream-api.md`; for the domain vocabulary see `CONTEXT.md`.

The mental model in one paragraph

One deep module, `PolestarClient`, knows everything about talking to Polestar: how to resolve which car a call means, how to get and cache a token, which headers to send, how responses are wrapped, how errors are shaped, and what "this car reports nothing here" means. A transport chain under it keeps the 10,000-calls-per-day budget honest: a cache serves repeats locally, a retry wrapper handles the API's transient failures, and every live attempt is metered before it goes out. On top sit table-driven domain tools, a handful of aggregate and planning tools that reuse the same data, an opt-in history store that turns the stateless API into a memory, and a humanizer that renders everything in short prose. The whole surface is available over stdio or stateless Streamable HTTP from the same engine.



What each module does

config.ts: the configuration registry

`loadConfig()` is the only place environment variables become a `Config` object, and the file also exports `KNOWN_ENV_VARIABLES`, the exact list the server reads. Three behaviors fall out of having a registry:

- **Unknown variables are reported, not ignored.** A typo in a variable the server never reads is otherwise invisible: the value is simply absent and behavior silently stays at its default. The server warns on startup and suggests the closest real name; `POLESTAR_STRICT_ENV=1` makes it fatal.
- **Invalid values fail before any request.** A base URL that is not an absolute URL, a non-numeric timeout, an out-of-range port, or an unsupported unit raises `InvalidConfigError` with the variable name, the problem, and the fix.
- **The Account ID question is asked, not guessed.** The gateway answers 403 `AUTHZ_CLIENT_ID_MISMATCH` when `x-client-id` carries the OAuth client ID, so live mode requires `POLESTAR_ACCOUNT_ID` explicitly, and the server warns when the resolved Account ID is identical to the client ID.

The dotenv loader here handles the idioms hand-rolled parsers get wrong: a leading `export`, values quoted with either kind of quote, and trailing `#` comments, while a `#` inside a quoted value stays data.

`server.ts`: wiring, stdio, and HTTP

`buildEngine()` constructs everything shared: config, budget meter, optional history store and sampler, the transport chain, token provider, and client. `createMcpServer()` registers the tools, resources, and prompts on an MCP server whose `instructions` field tells clients the important ground rules up front: the API is read-only, start with `list_vehicles`, prefer `get_car_status` over five domain calls, read the data age, and quote `requestId` when reporting problems.

Transport choice and modes:

- `stdio` (default) connects a `StdioServerTransport`; `stdout` carries the JSON-RPC protocol and nothing else, so diagnostics go to `stderr` and an unhandled rejection is logged rather than fatal.
- `POLESTAR_HTTP_PORT` switches to a stateless Streamable HTTP server: each request gets a fresh MCP server instance over one shared engine (cache, budget, history included). It binds `127.0.0.1` unless told otherwise, and refuses to bind any non-loopback address without `POLESTAR_HTTP_TOKEN`, because the server reads vehicle location and an unauthenticated listener would put that on the network. Subscriptions are a `stdio` feature.
- In fixture mode the secrets file is not read at all, so offline use never touches a live credential file.

Shutdown is deliberate: on `SIGTERM/SIGINT` the sampler stops taking quota calls, the MCP session closes so clients see a real end-of-session, and the exit code says "stopped" rather than "crashed" (with a 5-second hard exit as the backstop).

`domain-tools.ts`: the domain table

The heart of the tool surface is `DOMAINS`, one entry per read domain of the API: kind (`telemetry` or `charging`), URL path segment, OAuth scope, and a description. Everything else derives from that table: tool names (`target-soc` becomes `get_target_soc`), tool registration with a zod schema (`vin` optional, `raw` optional), the deduplicated scope list the token request sends, and the resource URI space. `list_vehicles` is registered separately because it is discovery over the credential itself.

`tool-output.ts` and `format.ts`: the humanizer

`readDomainFormatted()` is the one rendering path for domain reads: resolve the VIN, fetch through the cache, and answer with a header (vehicle, domain, data age), a humanized summary, and the raw payload when `raw: true` is requested. `format.ts` does the summarizing: enum labels lose their `CHARGING_STATUS_V2_`-style prefixes and gain readable wording ("Unplugged", "Asleep / not in use"), values get sane units, and each answer's age comes from the payload's own timestamp. A 403 answer gains a scope hint, because the M2M credential is scope-scoped per domain and the raw API message never says which scope was missing.

`aggregate-tools.ts`, `advisor-tools.ts`, `system-tools.ts`

- The four aggregate tools answer the questions people actually ask in one call, over data the domain tools already serve. They read through the cache, so an aggregate costs at most one live call per constituent domain and usually zero. `get_car_status` reads five domains in parallel; `is_car_secure` turns the exterior domain into a true/false verdict that names anything open; `get_needs_attention` filters the health domain down to active warnings; `get_charging_estimate` combines battery, target, and amp limit, states every assumption it makes, and falls back to the car's own time-to-full estimate when the data for exact math is missing.

- `plan_cheapest_charge` is the read-only smart-charging advisor: the caller supplies hourly prices as an argument, the tool adds the car's SoC, target SoC, amp limit, and charge window from `global-charge-timer`, and `planner.ts` computes the cheapest contiguous hours that reach the target (falling back to cheapest individual hours, and saying so, when the window is too short). No writes, no external fetches.
- `polestar_status` is the server's self-observation: budget used and remaining with its reset time, cache hit rate, and token expiry. An agent that can see the budget can pace itself instead of tripping the fail-closed guard.

`history.ts`, `sampler.ts`, `history-tools.ts`: memory for a stateless API

The API answers only "what is true right now", so any longitudinal question needs a local store. `HistoryStore` is an append-only JSONL file per vehicle and domain, deduplicated by the cloud's `metaEventId`: the cloud re-serves the same ID until the car reports something new, so unchanged snapshots cost a single line. A torn last line (a crash mid-append) is tolerated on read. Recording hooks into the cache's live-response callback, so passive recording never spends an extra call. `HistorySampler` optionally polls the four fast-moving domains on a timer (default every 10 minutes, clamped 60 to 3600 seconds) and stands down entirely once 80% of the daily budget is spent; it is off unless `POLESTAR_SAMPLE=1`. The three history tools ride on top: raw samples, a degradation series (implied full-charge range from range/SoC, implied capacity from usable energy/SoC; samples under 5% SoC are skipped as noise), and reconstructed charging sessions (a maximal run of charging samples, gaps up to 30 minutes allowed).

`client.ts`: the deep module

`PolestarClient` exposes `vehicles()`, `domain(vin, kind, name)`, and `resolveVin()`:

- `resolveVin()` turns the optional `vin` argument into a concrete car: an omitted VIN means "the only vehicle" (the personal-credential case); a credential with several cars throws `AMBIGUOUS_VIN` listing the VINs; a malformed one throws `INVALID_VIN` (17 characters, no I/O/Q). At the tool layer the ambiguity is caught and turned into an MCP elicitation question ("This credential can see N vehicles. Which one?") when the client can answer; the reply is passed through `resolveVin` again, so untrusted client input is still validated. Delegated accounts follow the same pattern: a call may pass `delegated_account_id`, which is checked against the `POLESTAR_DELEGATED_ACCOUNT_IDS` allowlist and refused (`DELEGATION_NOT_ALLOWED`) if absent, because the API silently accepts unknown delegated IDs.
- `domain()` sends `Authorization: Bearer, x-client-id` (plus `x-delegated-account-id` when configured), unwraps the `{data, meta}` envelope, and maps the one special case of a 404 `DATA_NOT_AVAILABLE` to `null`. Every other failure becomes a `PolestarApiError` carrying the API's own code, message, `httpStatus`, and, when present, `requestId`, `timestamp`, and `details`.
- `assertIdentity()` checks the envelope (and the payload's own VIN field) against the requested car and domain, refusing a response labelled for a different vehicle with `VIN_MISMATCH` or `DOMAIN_MISMATCH`. Presenting one car's telemetry as another's is the one error this server must never make, whether the mislabel comes from a bad fixture or an upstream surprise.
- A 401 triggers exactly one fresh-token retry; a second 401 surfaces as an error instead of burning the rate limit forever.

`auth.ts`: the token provider

`TokenProvider` owns the client-credentials flow: a POST to the token URL with a JSON body of `clientId`, `clientSecret`, and the space-joined scope list (overridable with `POLESTAR_SCOPES`). The token is cached until `expiresIn` minus a five-minute skew, concurrent callers share one in-flight request, `invalidate()` drops the cache but leaves the in-flight promise alone, and `expiresAt()` lets the status tool report how long the current token is still trusted. Token failures raise `AuthError` with the HTTP status and, when present, `requestId` and `timestamp`, reading both of Polestar's documented error shapes.

The transport chain

`Transport` remains a one-method seam (`fetch(url, request)` to status, headers, and body), and the chain layers three adapters around it:

- `CachingTransport` serves repeated GETs locally. Each URL gets a TTL from how fast the value can actually change: vehicle list 1 hour; battery, location, and exterior 30 seconds; availability, odometer, and parking climatization 60 seconds; pre-cleaning 5 minutes; health 10 minutes; charging settings 5 minutes. Once an entry expires, the caller gets the last known value immediately (counted as "stale served") while a single-flight refresh runs in the background, so latency never waits on Polestar. The token POST and error responses pass straight through, and every successful live domain GET fires the history recorder.
- `RetryingTransport` retries exactly the failures the API documents: 429 (honoring `Retry-After` when present) and 502/503, with exponential backoff capped at 5 seconds and at most 3 attempts. Every attempt, including retries, is metered against the budget before it goes out. Network errors are not retried; the caller sees them immediately.
- `HttpTransport` (a 15-second default deadline per request) and `FixtureTransport` (the captured-dump adapter) sit at the bottom, unchanged in spirit: the deadline exists because one hung connection would otherwise wedge every caller queued behind it, and the fixture adapter keeps missing fixtures (`NOT_FOUND`) deliberately distinct from missing data (`DATA_NOT_AVAILABLE`), replays captured error statuses, and refuses lookups that escape the fixtures directory.

In fixture mode the retry layer is skipped (replays are free) and the cache is disabled unless history is on, so a static dump cannot cache-dedupe anything useful.

`budget.ts`: the fail-closed meter

`BudgetMeter` counts actual live calls against the documented 10,000-per-client-per-day limit over a rolling UTC day window. Cache hits, stale re-serves, and fixture reads never consume budget. When the limit is reached, `consume()` throws `BudgetExceededError` before the request goes out: the server fails closed rather than hammering an API that will only answer 429. The status tool and the resource poller both read the same meter (subscriptions pause at 90% usage; the sampler stands down at 80%).

Error model

Error class	Raised by	When	Carries
MissingConfigError	config.ts	live mode without client ID, secret, or Account ID	the missing variables and every way to fix them
InvalidConfigError	config.ts	a variable is present but unusable	variable, problem, guidance
AuthError	auth.ts	the token endpoint failed or answered malformed	status, requestId, timestamp
TransportError	transport.ts	deadline exceeded or network failure	causeKind: timeout or network
BudgetExceededError	budget.ts	daily limit reached; fails closed	used, limit, reset time
PolestarApiError	client.ts	API error envelope (except DATA_NOT_AVAILABLE), unusable body, INVALID_VIN, AMBIGUOUS_VIN, NO_VEHICLES, VIN_MISMATCH, DOMAIN_MISMATCH	code, message, httpStatus, requestId, timestamp, details
null (not an error)	client.ts	404 with code DATA_NOT_AVAILABLE	tools turn this into a plain sentence

Design decisions, and why

- **A budget meter, not polite suggestions.** The API's only hard resource is the daily call allowance. Metering before every attempt and failing closed turns "the server exceeded the rate limit" into "the server stopped and said why".
- **Stale-while-revalidate over blocking refresh.** Agents ask questions; waiting 15 seconds for a fresh value they did not need fresh is worse than an instant answer with a visible age. Every answer carries its data age precisely so staleness is a known quantity, not a surprise.
- **History is opt-in and deduplicated.** Recording is passive (a cache hook), the sampler is a separate flag, and `metaEventId` deduplication keeps the store proportional to actual change. The degradation report presents series, not verdicts, because range is conditions-dependent.
- **One deep module, thin tools.** Token logic, headers, envelope handling, identity checks, and error rules are easy to get subtly wrong; keeping them in `PolestarClient` means the tools are a boring layer, and there is exactly one place for each API behavior.
- **The transport seam, not a fetch mock.** With real adapters, tests and offline use run the same code path as production; there is no separate "test mode" branch anywhere in the client.
- **Fixture honesty.** A missing fixture must never read as "the vehicle reports nothing", and fixture mode must never touch a live credential file. Both choices keep the null rule and the auth lifecycle falsifiable in tests.
- **Read-only by construction.** No tool mutates state because the documented v1 API publishes no write endpoints. `POLESTAR_ENABLE_WRITES` registers nothing today; it is the deliberate opt-in future write tools will slot behind (see `WRITE-ENDPOINTS.md`).

- **HTTP mode with a loopback guard.** The same engine serves stdio and HTTP, but a server that reads vehicle location must not come up unauthenticated on a shared network, so the non-loopback bind requires a bearer token.

The upstream API contract

Who this is for: developers who want to understand what the server is talking to, debug a live error, or build their own client. This page records the parts of the Polestar Data Portal M2M API that this implementation actually depends on. The domain vocabulary is in `CONTEXT.md`; what each endpoint returns in practice is in `tools-reference.md`.

What the M2M API is

The Data Portal M2M API is Polestar's machine-to-machine developer API (part of the EU Data Act developer portal). It is different from the API the Polestar phone app uses: the app talks to a consumer backend (GraphQL operations such as `GetConsumerCarsV2` and `CarTelematicsV2`, authenticated through `polestarid.eu.polestar.com`), while the M2M API is a credential-based REST API intended for the vehicle owner to share their car's data with services. The captured dump in the parent directory contains both kinds of traffic; this server implements only the M2M side.

Base URL and shape

- Base URL: `https://pc-api.polestar.com/eu-north-1/data-portal/m2m`
- Every documented operation is a GET. The v1 API is read-only.
- Vehicle routes are addressed by VIN, 17 characters, case-insensitive in the path.
- Rate limit: 10,000 calls per client per day. A client that polls in a loop will exhaust this; the server only ever makes calls in direct response to a tool call.

Authentication

The token endpoint is `POST {base}/token` with a JSON body:

```
{
  "clientId": "<client id>",
  "clientSecret": "<client secret>",
  "scope": "pdp-telemetry/battery pdp-charging/targetSoc ..."
}
```

The scope is optional on the wire. This server always sends the deduplicated union of its 15 domain scopes (see the scope column in the tool table), so one token covers every tool call. The response carries at least `accessToken` and `expiresIn` (seconds); the client caches it until `expiresIn` minus a five minute skew.

Token-endpoint errors come back in one of two shapes, and the client reads both:

- Polestar-style: { "error": { "code": "...", "message": "...", "requestId": "...", "timestamp": "..." } }
- OAuth-style: { "error": "invalid_client", "error_description": "..." }

Request headers

Header	Sent when	Meaning
authorization	always	Bearer <accessToken>
x-client-id	always	The Account ID from the Data Portal page. This is the caller identity, and it is not the OAuth client ID
x-delegated-account-id	third-party credentials only	Selects the account whose shared VINs you access when you were given credentials by someone else
accept	always	application/json

The Account ID / client ID distinction is the most common setup confusion. The OAuth client ID and client secret authenticate the credential at the token endpoint; the Account ID identifies whose data you are asking for on every vehicle route afterwards. The Data Portal UI shows both.

Response envelope

Every domain response is wrapped:

```
{
  "data": { "...": "the domain payload" },
  "meta": { "domain": "battery", "vin": "YSMTEST22PL000001" }
}
```

`meta` names the domain and the VIN it was served for. The client unwraps the envelope, so MCP tools return only the payload.

The availability model

Fields are reported independently: a car that has no washer-fluid sensor simply omits the field, and a car that reports nothing at all for a domain answers with an error instead. The documented rule:

- A domain with no data returns HTTP 404 with error code `DATA_NOT_AVAILABLE`. This is a normal answer for domains a vehicle does not support, most often `is-at-charge-location`.
- The client models that single case as `null` (tools render it as a plain sentence), and treats every other 4xx/5xx as a real error.

Error shape

All API errors share one envelope:

```
{
  "error": {
    "code": "INVALID_REQUEST",
    "message": "Invalid VIN format.",
    "httpStatus": 400,
    "requestId": "req-42",
    "timestamp": "2026-01-01T00:00:00Z",
    "details": { "requested_vin": "YSMTEST22PL000001" }
  }
}
```

`requestId` and `timestamp` are the useful ones when reporting a problem: they are what Polestar support can look up. The server preserves them on `PolestarApiError` and on `AuthError`.

Endpoints used

Route	Purpose
POST /token	Client-credentials token for all scopes
GET /v1/vehicles	VINs the credential may access
GET /v1/vehicles/{vin}/telemetry/{availability,battery,exterior,health,location,odometer,parking-climatization,pre-cleaning}	The eight telemetry domains
GET /v1/vehicles/{vin}/charging/{amp-limit,charge-locations,charge-now,global-charge-timer,parking-climate-timer,target-soc,is-at-charge-location}	The seven charging domains

Write scopes without write endpoints

Tokens can carry write-flavored scopes such as `pdp-charging/targetSoc` and `pdp-charging/overrideChargeTimer`, and the read endpoints documented here use those same scope names. What the sandbox documentation does not publish is any v1 write endpoint: there is no documented way to set a target state of charge or override a charge timer over the M2M API today. The server therefore requests the union of its domain scopes (overridable with `POLESTAR_SCOPES` for a least-privilege token) and implements only reads. The full design for the day writes arrive, including the safety rules that are non-negotiable, is in `WRITE-ENDPOINTS.md`.

What the server adds on top of the contract

The API is a bare request/response surface; the server adds the operational discipline a 10,000-calls-per-day credential needs. These behaviors are the server's own, not the API's:

- **Identity assertions.** The envelope labels each response with its VIN and domain, and payloads repeat the VIN. The client refuses a response labelled for a different vehicle or domain (`VIN_MISMATCH`, `DOMAIN_MISMATCH`) rather than presenting one car's telemetry as another's.
- **Retry with limits.** 429 responses (honoring `Retry-After` when present) and 502/503 gateway errors are retried up to three times with capped exponential backoff; every attempt, including retries, is metered against the daily budget before it goes out.
- **Caching.** Repeated reads are served locally with a per-domain freshness policy and stale-while-revalidate, so latency never waits on Polestar and the budget is spent only on genuinely new information.
- **Fail-closed budget.** When the daily allowance is spent, the server refuses to send further calls (with the reset time in the error) instead of hammering an API that will only answer 429.

Development and verification

Who this is for: developers working on the server itself. Setup for users is in `getting-started.md`, the design rationale in `architecture.md`.

Repository layout

```

polestar-mcp/
src/
  server.ts          wiring: config, engine, MCP surface, modes
  config.ts          configuration registry, validation, dotenv loader
  client.ts          PolestarClient: VINs, envelope, identity, errors
  auth.ts            TokenProvider: client-credentials lifecycle
  domain-tools.ts    DOMAINS table, vehicle/domain lists + 15 get_*
  aggregate-tools.ts car status, security, attention, charge estimate
  advisor-tools.ts   plan_cheapest_charge (prices in, schedule out)
  system-tools.ts    polestar_status (budget, cache, token)
  history-tools.ts   history/degradation/sessions (opt-in)
  tool-output.ts     shared plumbing: vin/raw args, 403 scope hint
  format.ts          humanizer: enum labels, units, ages, summaries
  planner.ts         cheapest-hours arithmetic for the advisor
  history.ts         JSONL store, sessions, degradation series
  sampler.ts         optional background sampler, budget guard
  budget.ts          fail-closed daily-call meter
  caching-transport.ts TTLs, stale-while-revalidate, history hook
  retrying-transport.ts 429/502/503, Retry-After, backoff
  transport.ts       the seam: HttpTransport, FixtureTransport
test/
  auth.test.ts       token lifecycle
  client.test.ts     envelope, headers, VINs, identity, errors
  runtime.test.ts    cache, retry, budget behavior
  planner.test.ts    charge-window math including wrap-around
  config-registry.test.ts variables, validation, unknown detection
  helpers.ts         ScriptedTransport for scripted responses
  fixtures/dump/     a small sanitized dump for tests
scripts/
  smoke.ts           stdio end-to-end: registry, tools, resources
  smoke-http.ts      Streamable HTTP end-to-end over POSTs
  demo-dump.ts       stdio checks against the real captured dump
  stdio-client.ts    minimal MCP stdio test client
  check-secrets.mjs  the secret-hygiene gate
docs/
  GATES.md           this documentation set (gates: docs/GATES.md)
  CONTEXT.md         the six build/verification gates
  the domain vocabulary

```

Commands

Command	What it does
<code>npm install</code>	Installs dependencies (MCP SDK v2, zod; TypeScript and tsx for development)
<code>npm run build</code>	Strict TypeScript compile into <code>build/</code> , the endpoint your MCP client runs
<code>npm start</code>	Runs <code>build/server.js</code> (stdio, or HTTP when <code>POLESTAR_HTTP_PORT</code> is set)
<code>npm run dev</code>	Runs <code>src/server.ts</code> directly through tsx, no build step
<code>npm test</code>	The unit suites through the Node test runner (TAP output)
<code>npm run smoke</code>	Full stdio integration test against the sanitized fixtures
<code>npm run smoke-http</code>	Full Streamable HTTP integration test
<code>npm run demo-dump</code>	Full stdio integration test against the real captured dump one level up
<code>npm run check-secrets</code>	The secret-hygiene gate
<code>npm run typecheck</code>	Types src, test, and scripts together with no emit

Node 20 or newer is required (the engine field enforces it). The production build targets ES2022 with strict settings turned up: `strict`, `noUncheckedIndexedAccess`, and `exactOptionalPropertyTypes`. The dev typecheck covers `test/` and `scripts/` under the same settings, so no source file is exempt from the declared contract.

What the unit tests prove

The suites run through a `ScriptedTransport` that hands out pre-written responses and records every call, so each test can assert both the outcome and the exact wire behavior.

- `auth.test.ts`: concurrent `token()` callers collapse into a single request; a cached token is reused; refresh happens only after `expiresIn` minus the five minute skew and the request body carries the exact scope list; a token-endpoint failure surfaces as an `AuthError` with the OAuth error text, `requestId`, and `timestamp`.
- `client.test.ts`: envelope unwrapping; the exact headers (`authorization`, `x-client-id`, `accept`, and `x-delegated-account-id` when configured); VIN resolution (omitted with one vehicle, ambiguous with several, malformed rejected); identity checks that refuse a response labelled for a different vehicle or domain; error normalization preserving `code`, `httpStatus`, `requestId`, and `details`; the 404 `DATA_NOT_AVAILABLE` to `null` rule; exactly one fresh-token retry on 401.
- `runtime.test.ts`: cache TTL and stale-while-revalidate behavior (expired entries serve the last value and refresh single-flight in the background; POSTs and errors bypass the cache); retry behavior (429 honoring `Retry-After`, 502/503 backoff, non-retryable errors pass through); budget metering including the fail-closed stop.
- `planner.test.ts`: the cheapest-hours math, including a charge window that wraps midnight, unpriced slots treated as last resorts, and the contiguous-block fallback.

- `config-registry.test.ts`: the variable registry, validation failures (bad URL, non-numeric numbers, out-of-range ports, unsupported units), and unknown-variable detection with suggestions.

End-to-end scripts

`scripts/stdio-client.ts` is a minimal MCP client: it spawns the server as a child process, speaks newline-delimited JSON-RPC on stdio, and offers typed helpers for the handshake, `tools/list`, `tools/call`, and resource reads. Both stdio test scripts are built on it.

`npm run smoke` starts the server in fixture mode against the sanitized test fixtures and asserts, in order: the handshake completes with the expected server name, the tool registry is complete (every domain tool plus the aggregate, advisor, system, and history tools), tools carry their `readOnlyHint` annotations, the optional-vin resolution works, `get_car_status` produces a humanized briefing, resources list and read, the three prompts exist, the history recorder feeds `get_history`, a `DATA_NOT_AVAILABLE` domain degrades into a plain sentence, and an unknown tool is rejected cleanly. Only after every assertion does it print `SMOKE_OK`.

`npm run smoke-http` starts the server with `POLESTAR_HTTP_PORT` set and speaks plain JSON-RPC over HTTP POSTs (stateless mode): handshake, tool registry, a `get_battery` call, and a resource read. It prints `HTTP_SMOKE_OK`.

`npm run demo-dump` runs the stdio checks against the real captured dump in the parent directory (the `api_endpoints` folder this project lives in), which proves the fixture layout matches reality. It prints `DUMP_OK` on success and never prints VINs or data values beyond existence checks.

The verification gates

Completion of this project is tracked in two ledgers, and each gate records its own evidence (exit status, output fingerprint) next to its `CHECK` line:

- `GATES.md` (the server): build, unit suites, stdio smoke, secret hygiene, real-dump compatibility, and the HTTP smoke.
- `docs/GATES.md` (this documentation set): tool and environment-variable parity against the source, link integrity, writing hygiene, secret hygiene with docs in the scanned tree, and the rendered PDF handbook.

The ledgers are read by the unlazy gate checker, which refuses to execute a `CHECK` line that has not been explicitly approved, and records evidence so a checked box without evidence counts as unmet. To re-verify everything after changes:

```
node /path/to/unlazy/scripts/gate-check.mjs --reverify \
  --root . --cwd . GATES.md docs/GATES.md
```

Read the ledgers for current status rather than trusting any summary.

Secret hygiene

Credentials live outside the project tree by design (the search order is in `getting-started.md`). The scanner in `scripts/check-secrets.mjs` enforces the other half of the discipline:

- A credential file inside the tree is itself a finding, whatever it holds. The rule is a pattern, not a list of names: `.env.production` holds a live token just as well as `.env.secrets` does, so anything matching the `.env` family (or an `env/secrets/credentials` suffix) is flagged. `.gitignore` ignores the whole `.env` family, and the scanner catches files git would track.
- Three structural detectors run over every text file (skipping `node_modules`, `build`, `.git`, `images`, and source maps): three-segment JWT shapes, opaque lowercase credential blobs of 25 to 40 characters (the shape Apigee consumer keys take), and any 17-character VIN-shaped identifier.
- Detectors match shapes, never values. A scanner that embedded the owner's real VIN or client ID would become the leak it exists to catch, and its output reports only file and line, never matched text.
- Every detector must first fire on a synthetic positive control, or the scan exits with `CONTROL_FAIL` and a clean result means nothing.
- The one allow-listed VIN shape is the synthetic test marker `YSMTEST...` (regex `^YSMTEST\d{2}PL\d+$`), which cannot be a real Polestar identifier. Every other VIN-shaped match is treated as genuine.

Treat captured dumps like passwords. The dump in the parent directory contains long-lived refresh tokens and location history; do not commit it, email it, or paste its contents into issues.

Adding a tool

Domain tools are table-driven: a new read domain is one new row in the `DOMAINS` table in `src/domain-tools.ts` (kind, URL path segment, OAuth scope, description). The tool name, registration, token scope, resource URI, and fixture layout follow automatically:

1. Add the row, plus a fixture file at the matching path under `test/fixtures/dump/dataportal/` wrapped in the `{data, meta}` envelope (an error envelope with `"httpStatus": 404` and code `DATA_NOT_AVAILABLE` tests the missing-data path).
2. If the payload needs humanizing, add a `summarizeDomain` case in `src/format.ts`; without one, tools still answer correctly with the raw payload.
3. Extend the unit tests for new wire behavior; the smoke script derives its expected registry from the code.

A tool outside the domain table (like the aggregates) lives in its own module with a `register*Tools(server, deps)` function that `createMcpServer` calls, its own zod input schema, and the `readOnlyHint` annotation. A write tool (if Polestar ever publishes v1 writes) would additionally require `POLESTAR_ENABLE_WRITES`, an explicit `vin` argument, and a `destructiveHint` annotation; the full safety design is in `WRITE-ENDPOINTS.md`.

Versioning

The server reports its version in the MCP handshake from `package.json` (currently 0.3.0), so there is exactly one version number to bump.

Live contract findings: Polestar Data Portal M2M

Measured 2026-09-18T18:41Z against `https://pc-api.polestar.com/eu-north-1/data-portal/m2m` with the owner's own M2M credential and single authorized vehicle. VINs masked, coordinates rounded. These replace the guesses in `BACKLOG-TABLE.md` rows Q1-Q6, D7 and E7.

Identity headers (settles E7)

Request	Result
<code>x-client-id</code> = Data Portal Account ID	200
<code>x-client-id</code> = OAuth clientId (the old <code>config.ts</code> fallback)	403 AUTHZ_CLIENT_ID_MISMATCH: "The x-client-id header does not match the identity associated with this token."
<code>x-client-id</code> omitted	400 VALIDATION_INVALID_PARAMETER
<code>x-delegated-account-id</code> = a value that does not exist	200 , silently ignored

Consequences: the Account ID is a separate identifier and is **required**; the client-ID fallback was broken by construction. The Account ID observed here equals the Polestar ID subject identifier (`sub`) from the consumer ID token, i.e. a UUID, not an email. An invalid delegated account is accepted without error, so delegation cannot be validated by probing: it must be reported in `check_credential` and never assumed to have taken effect.

VIN handling (settles Q3)

Request	Result
syntactically valid VIN the credential cannot see	403 AUTHZ_VIN_UNAUTHORIZED with <code>details.requested_vin</code>
malformed VIN (<code>short</code> , 5 chars)	403 AUTHZ_VIN_UNAUTHORIZED: the gateway does not validate format

So VIN format checking exists only where we put it (the tool input schema), and any unauthorized or typo'd VIN reads as an authorization failure. The hint for `AUTHZ_VIN_UNAUTHORIZED` must mention both cases.

Domain list (settles Q5 partially)

Request	Result
<code>/v1/vehicles/{vin}/telemetry</code> <code>/not-a-real-domain</code>	404 <code>VALIDATION_RESOURCE_NOT_FOUND</code> : "Unsupported telematics domain: ..."

`VALIDATION_RESOURCE_NOT_FOUND` is not `DATA_NOT_AVAILABLE`. A mistyped domain is a contract error, not an empty report; treating the two alike would hide a bug in our own routing.

Live domain scan, single authorized vehicle

All 15 domains answered 200 except `is-at-charge-location` (**404** `DATA_NOT_AVAILABLE`). Key counts and the metadata-only domains confirm the captured dump was not stale:

- `charge-locations` → 5 keys: `vin`, `id`, `utc0`, `metaReceivedAt`, `metaEventId`. **No locations at all, live.**
- `charge-now` → 5 keys, the only content is `syncedOverrideChargeTimer.updatedAt`. **No boolean state, live.**
- `battery` → 14 keys; no volts/amps/watts, no since-charge or trip consumption, no breakdown, live.
- `availability` → `AVAILABILITY_STATUS_AVAILABLE` + `USAGE_MODE_ABANDONED`, reported 15 minutes before the probe.
- `location` → `coordinate.latitude/longitude` **numbers**, `altitude` and `speed` **strings**, `heading` **number**.

`utc0: true` appears on `charge-locations` and `parking-climate-timer` only; `global-charge-timer` carries an explicit `timeZone.offsetMinutes` per boundary instead. The gateway gives no field that disambiguates `utc0`, so the server renders the raw hour **and** the offset and never invents a single interpretation.

Error envelope shape

`{ error: { code, message, httpStatus, requestId, timestamp, details? } }`, confirmed. Two gotchas: `message` is itself an **escaped JSON string** for `VALIDATION_INVALID_PARAMETER`, and timestamps in error envelopes are ISO-8601 while payload timestamps are protobuf `{seconds:"...",nanos}`.

Token endpoint

`POST /token` with `{clientId, clientSecret}` and **no** `scope` still returns a token that reads all 15 domains; `tokenType` is `Bearer` and `expiresIn` is 3600. Requesting the explicit scope union is therefore a forward-compatibility choice, not a requirement, and a smaller scope set is available to callers who want least-privilege tokens.

No published contract at the gateway

`openapi.json`, `swagger.json`, `api-docs/`, `/v1` and `/` all answer the Apigee "Missing Authentication Token" 403 (i.e. no such route), and `data-portal.polestar.com` redirects to login. There is no machine-readable spec reachable programmatically, so field-level truth for this project is: the captured dump plus live probes, pinned by fixture schema tests (row F12).

Consumer API adapter design note (not implemented)

The captured dump proves the Polestar **consumer app API** yields three things the official M2M API does not:

1. **Car identity:** GraphQL `GetConsumerCarsV2` returns model name/year, registration number, structure week, and the `pno34` spec code that encodes trim, motor, and pack (this car: `534EDPP0E11972900RFA000...XPLUSS`, a 2023 Polestar 2 long range dual motor, XPLUSS package).
2. **Car renders:** GraphQL `GetCarImages` returns 6 angles of the exact specced car, opaque JPG + transparent PNG, from `car-images.polestar.com`.
3. **App-side telematics:** `CarTelematicsV2` mirrors M2M battery/health/ odometer (kept as fallback only).

Why it is not wired in

- **Unofficial and undocumented.** The GraphQL query documents are not published; the dump holds responses, not requests. Reconstructing queries without the schema would be guesswork that breaks silently.
- **Auth rotates hard.** Polestar ID access tokens live ~299s and each refresh issues a NEW refresh token (rotation). A safe adapter must persist the rotated refresh token atomically on every refresh: lose it once and the car is locked out until re-login. The M2M credential has no such failure mode.

Shape when it lands

- `ConsumerAuthProvider` implements the same `TokenProvider` interface (refresh-token grant against `polestarid.eu.polestar.com/as/token.oauth2`), persisting rotation to a file under `~/.config/polestar-mcp/`.
- A `ConsumerTransport` speaks GraphQL behind the same `Transport` seam.
- Tools: `get_car_details` (decode `pno34` → trim/pack), `get_car_images` (returns the 6 render URLs), gated behind `POLESTAR_CONSUMER_REFRESH_TOKEN` env. Data feeds `get_car_status` extras (registration number, pack size) and the planner's `capacityKwh` default.

Write endpoints (designed, waiting for Polestar)

The M2M token already carries the write scopes (pdp-charging/targetSoc, pdp-charging/overrideChargeTimer, pdp-charging/ampLimit, pdp-charging/globalChargeTimer, pdp-charging/parkingClimateTimer), but the documented v1 API publishes **no write endpoints**: every operation is a GET. This server therefore ships read-only, and POLESTAR_ENABLE_WRITES=1 currently registers nothing; it exists so the day Polestar documents writes, the tools land behind the deliberate opt-in that is already wired.

What slots in where

Every expected call is a PUT /v1/vehicles/{vin}/charging/<name> (a guess, unconfirmed); the table lists the <name> that would sit at that spot.

Scope	Expected (guess, unconfirmed)	Tool
pdp-charging/targetSoc	target-soc	set_target_soc {vin, percent}
pdp-charging /overrideChargeTimer	charge-now	start_charging / stop_charging
pdp-charging/ampLimit	amp-limit	set_amp_limit {vin, amps}
pdp-charging/globalChargeTimer	global-charge-timer	set_charge_window {vin, startHour, endHour, activated}
pdp-charging /parkingClimateTimer	parking-climate-timer	set_climate_timer {vin, ...}

Safety design (non-negotiable when writes land)

- Writes stay behind POLESTAR_ENABLE_WRITES=1; reads never ask for confirmation, writes always carry destructiveHint: true annotations so MCP clients prompt the user.
- Every write tool takes an explicit vin: no single-vehicle defaulting, no "the only car" guessing on mutating calls.
- PolestarClient gains writeDomain(vin, kind, name, body) next to domain(); writes bypass the cache and invalidate the entry they mutate.